



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

Smart Renewable Energy Monitoring and Grid Optimization Platform

A CASE STUDY REPORT

Submitted in the partial fulfilment for the Course of

CSA1016 – SOFTWARE ENGINEERING

to the award of the degree of

BACHELOR OF ENGINEERING

IN

COMPUTER SCIENCE AND ENGINEERING

Submitted by

NALLAGOLLA SAI HITHESH(192372245)

Under the Supervision of

Dr.ANITA DAVAMANI.K

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

ABSTRACT:

The rapid growth of web applications and Internet services has increased the number of malicious URLs used for phishing, credential theft, redirection and other cyber attacks. This project presents a lightweight URL-based attack identification system that examines URL and IP-related characteristics and assigns a risk score. The system extracts features such as HTTPS usage, IP-address based host names, URL length, special symbols, suspicious keywords, hyphens and subdomain depth. A Flask web application provides both a browser interface and a JSON API.

The technical assignment additionally demonstrates how Git and Docker can be used throughout the development lifecycle. Git provides version control, feature isolation, commit history and controlled integration. Docker packages the Python runtime, dependencies and application into a reproducible container. Docker Compose simplifies execution and provides a foundation for extending the application with databases, caches and monitoring services.

1. INTRODUCTION AND PROBLEM ANALYSIS:

1.1 Background:

A URL is one of the primary entry points through which a user reaches a web resource. Attackers can manipulate URLs to imitate trusted services, hide the actual destination, redirect users or host malicious login pages. A URL may contain signals that indicate abnormal behaviour, including direct IP-address hosts, excessive subdomains, suspicious words, unusual special characters and insecure HTTP communication.

The project focuses on identifying these signals from URL and IP-related input data. It is intended as an educational cybersecurity prototype rather than a replacement for enterprise threat-intelligence systems. The implementation uses deterministic feature extraction and a transparent scoring mechanism so that the reason for a classification can be understood.

1.2 Problem Statement:

Design and implement a system that accepts URL data, extracts security-related characteristics, detects suspicious patterns and reports a risk classification. The project must also be managed through Git and packaged with Docker so that the application can be developed collaboratively and executed consistently.

1.3 Scope:

URL parsing and feature extraction.

Detection of IP-address based hosts.

Identification of suspicious keywords and URL patterns.

Risk-score based classification.

Browser and API access.

Git-based version management.

Docker-based deployment.

2. OBJECTIVES AND REQUIREMENTS

2.1 Project Objectives:

Identify suspicious URL patterns using measurable features.

Provide an interpretable risk score rather than an unexplained binary result.

Develop a simple web interface for analysis.

Provide a JSON API for programmatic testing.

Maintain source code using a structured Git workflow.

Containerize the application for repeatable deployment.

Provide automated tests for important classification behaviour.

2.2 Functional Requirements:

The system shall accept a URL through a web form.

The system shall accept JSON requests through an API endpoint.

The system shall parse the URL into its components.

The system shall identify whether the host is an IPv4 address.

The system shall count suspicious keywords and selected URL characteristics.

The system shall calculate a risk score.

The system shall return a classification.

The system shall expose a health endpoint for deployment verification.

2.3 Non-Functional Requirements:

Usability: the interface should be simple and understandable.

Portability: the application should run consistently through Docker.

Maintainability: source code and configuration should be version controlled.

Testability: core classification behaviour should have automated tests.

Security: input should be treated as untrusted data and parsed safely.

Performance: URL analysis should complete quickly for normal requests.

3. SYSTEM ARCHITECTURE AND REPOSITORY STRUCTURE:

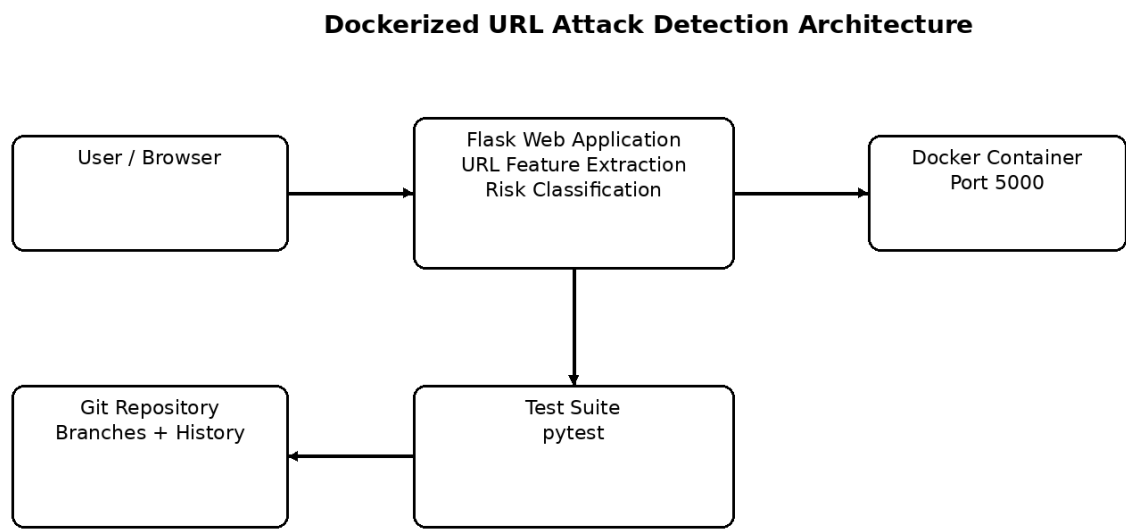


Figure 1. Overall Dockerized system architecture.

The user interacts with the Flask application through a browser or API client. The application extracts URL features and calculates a risk score. The application is executed inside a Docker container, while the test suite validates the classification logic. Git stores the complete source and configuration history.

3.1 Repository Structure:

Repository Folder Structure

```
URL-Attack-Detector/
├── src/app.py
├── tests/test_app.py
├── data/
├── docs/
│   ├── docker_architecture.png
│   └── git_workflow.png
├── screenshots/
├── Dockerfile
├── docker-compose.yml
├── requirements.txt
├── .gitignore
└── README.md
```

Figure 2. Project repository folder structure.

3.2 Major Files and Folders:

Item	Purpose	Importance
src/app.py	Flask application, URL parsing, feature extraction and scoring.	Core source
tests/test_app.py	Automated tests for benign and suspicious cases.	Quality
Dockerfile	Build instructions for the application image.	Deployment
docker-compose.yml	Service definition and port mapping.	Orchestration
requirements.txt	Pinned Python packages.	Dependencies
.gitignore	Excludes generated/local files.	Version control
docs/	Architecture, workflow and documentation.	Report evidence
screenshots/	Location for execution evidence.	Submission

4. GIT REPOSITORY INITIALIZATION:

4.1 Why Git Is Used:

Git records changes to project files and allows a development team to maintain multiple versions of the project. It supports experimentation through branches, review through commits and recovery through historical snapshots. For a cybersecurity project, traceability is particularly important because implementation changes should be attributable and reproducible.

4.2 Initialization Procedure:

```
mkdir URL-Attack-Detector
cd URL-Attack-Detector
git init
git status
git add .
git commit -m "chore: initialize URL attack detector"
git branch -M main
git status
```

4.3 Essential Git Files:

.git/ – hidden directory containing repository metadata, objects, references, index and configuration.

.gitignore – lists files that should not enter version control.

README.md – explains installation, execution and project purpose.

Working tree – current editable files.

Index/staging area – selected changes prepared for the next commit.

4.4 Remote Synchronization:

```
git remote add origin <repository-url>
```

```
git push -u origin main
```

```
git pull origin main
```

```
git push origin main
```

A remote repository allows the project to be synchronized between development machines and provides a shared location for collaboration.

5. GIT BRANCHING STRATEGY:

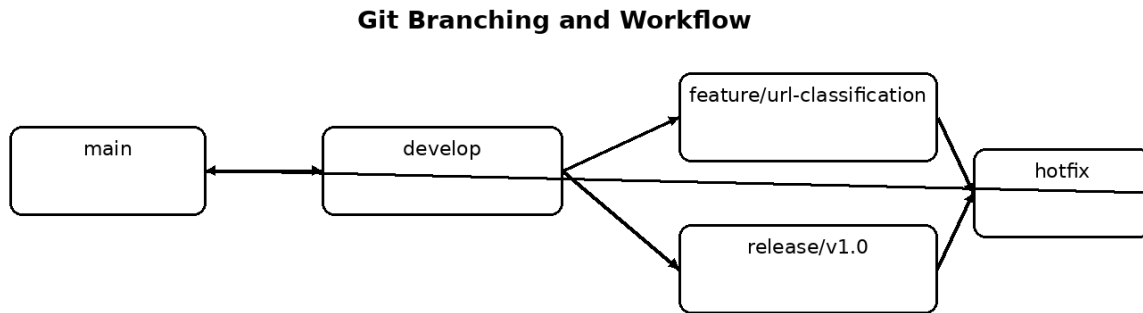


Figure 3. Recommended Git branching and integration workflow.

5.1 Branch Roles:

main: stable and submission-ready version.

develop: integration branch for completed features.

feature/*: short-lived branches for individual capabilities.

release/*: stabilization branch for final testing and documentation.

hotfix/*: urgent corrections to the stable release.

5.2 Feature Development Example:

```
git checkout develop
```

```
git checkout -b feature/url-classification
```

```
# implement feature
```

```
git add src/app.py tests/test_app.py
```

```
git commit -m "feat: add URL risk classification"
```

```
git checkout develop
```

```
git merge --no-ff feature/url-classification
```

```
git push origin develop
```

The branching model prevents incomplete feature work from directly changing the stable branch. It also creates a clear history showing which changes belong to each feature.

5.3 Branch Naming Rules:

Use lowercase names and hyphens.

Use feature/ for new functionality.

Use fix/ for defects.

Use docs/ for documentation.

Use release/ for stabilization.

6. VERSION CONTROL WORKFLOW:

6.1 Typical Development Cycle:

Pull the latest integration branch.

Create a feature branch.

Implement a small logical change.

Run tests.

Stage only relevant files.

Commit using a descriptive message.

Push the feature branch.

Merge after review/testing.

Delete the completed feature branch.

6.2 Conflict Resolution:

A merge conflict occurs when two branches modify overlapping parts of a file in incompatible ways.

Git marks the conflict so that the developer can decide which version should remain.

git checkout develop

git merge feature/url-classification

resolve marked sections in the files

git status

git add <resolved-files>

git commit -m "fix: resolve merge conflict"

git log --oneline --graph --all

6.3 Meaningful Commit Examples:

chore: initialize URL attack detector

feat: add URL risk classification

test: add URL classification tests

docs: add Git and Docker documentation

build: add Docker container configuration

fix: improve suspicious URL scoring

6.4 Good Commit Practice:

A good commit should contain one logical change, use a concise imperative message, avoid unrelated file modifications and leave the project in a usable state whenever possible.

7. COMMIT HISTORY AND REPOSITORY SYNCHRONIZATION:

7.1 Sample History:

a31f9c1 feat: add URL risk classification

5e18d2a test: add URL classification tests

9bc1a77 docs: add Git and Docker documentation

2c4e5f0 build: add Docker container configuration

0f1a8d2 chore: initialize URL attack detector

The history shows a progression from repository initialization to implementation, testing, documentation and containerization. The prefixes communicate the purpose of each commit and make the history easier to search.

7.2 History Commands:

git log

git log --oneline

git log --oneline --graph --decorate --all

git show <commit-id>

git diff

git status

7.3 Synchronization Workflow:

git fetch origin

git checkout develop

git pull origin develop

git push origin develop

git checkout main

git merge develop

git push origin main

Fetching updates remote-tracking information without changing the working tree. Pull combines fetching and integration. Push sends local commits to the remote repository.

7.4 Repository Maintenance:

Keep commits small and focused.

Delete merged feature branches.

Tag important releases such as v1.0.

Review changes before merging.

Keep README and deployment files synchronized with the code.

8. DOCKER ENVIRONMENT DESIGN:

8.1 Need for Containerization:

Python applications can behave differently when dependency versions or operating-system packages differ between machines. Docker solves this problem by packaging the runtime environment and application into an image. A container created from the image provides an isolated and repeatable execution environment.

8.2 Container Components:

Python 3.12 runtime.

Flask web framework.

Application source code.

Python dependency set.

Network port 5000.

Health endpoint used for basic deployment verification.

8.3 Docker Lifecycle:

Dockerfile

↓

docker build

↓

Docker image

↓

docker run / docker compose

↓

Running container

↓

Browser / API client

8.4 Advantages:

Consistent execution environment.

Easy setup on another computer.

Isolation from host dependencies.

Simple deployment and rollback.
Foundation for scaling and service separation.

9. DOCKERFILE DEVELOPMENT:

```
FROM python:3.12-slim

WORKDIR /app

ENV PYTHONDONTWRITEBYTECODE=1
ENV PYTHONUNBUFFERED=1

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY src ./src

EXPOSE 5000

CMD ["python", "src/app.py"]
```

9.1 Instruction Explanation:

Instruction	Explanation
FROM	Selects the Python 3.12 slim base image.
WORKDIR	Sets /app as the working directory.
ENV	Configures predictable Python runtime behaviour.
COPY	Copies dependency metadata and source code.
RUN	Installs Python dependencies inside the image.
EXPOSE	Documents the port used by the application.
CMD	Starts the Flask application when the container runs.

9.2 Image Build:

docker build -t url-attack-detector .
The Docker daemon reads the Dockerfile, downloads the base image if required, installs dependencies and creates a reusable image tagged url-attack-detector.

9.3 Image Verification:

docker images
docker inspect url-attack-detector

10. DOCKER COMPOSE DESIGN:

services:

url-attack-detector:

build: .

container_name: url-attack-detector

ports:

- "5000:5000"

environment:

FLASK_ENV: production

restart: unless-stopped

10.1 Service Explanation:

services defines the containers managed by Compose.

build: . tells Compose to build the image from the project directory.

container_name gives the running container a predictable name.

ports maps host port 5000 to container port 5000.

environment supplies runtime configuration.

restart: unless-stopped improves availability after accidental stops or host restarts.

10.2 Networking:

Compose creates an isolated network for its services. Although the current prototype has one service, this architecture can be extended with a database, Redis cache, reverse proxy or monitoring service without changing the basic deployment approach.

10.3 Volumes and Future Extension:

The current service does not require persistent storage because the prototype does not store analysis results. If persistent results are added later, a named volume can be attached to a database service. This avoids placing database files inside the application container.

docker compose up --build

docker compose ps

docker compose logs

docker compose down

11. CONTAINER DEPLOYMENT AND TESTING:

11.1 Build and Run:

```
docker build -t url-attack-detector .
```

```
docker run --rm -p 5000:5000 url-attack-detector
```

After the container starts, open the application in a browser at <http://localhost:5000>. The page displays the URL analysis form.

11.2 Health Test:

```
curl http://localhost:5000/health
```

Expected:

```
{"status":"ok"}
```

11.3 API Test:

```
curl -X POST http://localhost:5000/api/analyze ^  
-H "Content-Type: application/json" ^  
-d "{\"url\":\"http://192.168.1.10/login/verify\"}"
```

The response contains the submitted URL, calculated risk score, classification and extracted features. The example is designed to trigger several suspicious characteristics, including HTTP communication, an IP-address host and suspicious path keywords.

11.4 Automated Tests:

```
pip install -r requirements.txt
```

```
pytest -q
```

The automated test suite verifies a normal HTTPS domain and an IP-based suspicious URL. Docker execution should also be verified using the health endpoint and browser interface.

11.5 Evidence Screenshots to Capture:

git init and git status

git log --oneline --graph

branch creation and merge

docker build output

docker images

docker compose up

docker compose ps

browser showing the application

health endpoint response

API analysis response

pytest successful result

12. SECURITY CONSIDERATIONS:

The application processes untrusted URL input. The implementation therefore treats the input as data and uses a URL parser rather than executing any supplied content. The project should not visit submitted URLs automatically. This reduces the risk of turning the detector into an unintended URL-fetching mechanism.

12.1 Security Improvements:

Normalize and validate URLs more strictly.

Add domain reputation and threat-intelligence sources.

Use a labeled dataset and evaluate precision, recall and F1-score.

Add rate limiting to public API endpoints.

Run the container as a non-root user in a production deployment.

Scan images for known vulnerabilities.

Store secrets outside source control.

13. BENEFITS OF GIT AND DOCKER:

Collaboration: developers can work on isolated branches and merge controlled changes.

Version management: every meaningful change can be reviewed and reverted.

Traceability: commit history explains why and when implementation changes occurred.

Portability: Docker packages the application and dependencies.

Deployment consistency: the same image can be tested and deployed across environments.

Scalability: Docker Compose provides a foundation for adding services.

Maintainability: source, configuration, tests and documentation are kept together.

13.1 Combined Git + Docker Workflow:

A practical lifecycle is: create a feature branch, implement the change, run tests, build the Docker image, test the container, commit the verified change, merge into develop, perform release testing and finally promote the stable code to main. This connects version control evidence with deployment evidence.

14. CHALLENGES, IMPROVEMENTS AND FUTURE ENHANCEMENTS:

14.1 Challenges:

Dependency versions may differ between development environments.

Merge conflicts can occur when multiple branches change the same lines.

Docker builds can fail if dependency installation or file paths are incorrect.

Security classification based only on handcrafted rules can produce false positives or false negatives.

Actual deployment evidence must be captured from the student's environment.

14.2 Improvements:

Introduce a machine-learning classifier using a labeled URL dataset.

Add persistent storage for historical analysis.

Add a dashboard with statistics and trends.

Add CI/CD to automatically run tests and build Docker images.

Use image scanning and dependency vulnerability checks.

Add structured logging and monitoring.

Use HTTPS behind a production reverse proxy.

Introduce authentication and role-based access for administrative functions.

15. EXPECTED DELIVERABLES:

Deliverable	Included / Required Evidence
Technical report	This 15+ page DOCX report.
Git repository	Source code and complete commit history.
Source code	src/app.py and tests.
Dockerfile	Included in project package.
docker-compose.yml	Included in project package.
Git screenshots	Capture from student's own terminal.
Docker screenshots	Capture build, run and Compose execution.
Architecture diagrams	Generated and embedded in this report.
Workflow diagram	Generated and embedded in this report.

16. CONCLUSION:

The Identification of URL-Based Attacks from IP Data project demonstrates a complete development and deployment workflow using Git and Docker. The application extracts URL security features and generates an interpretable risk classification. Git provides controlled development, branching, commit history and synchronization, while Docker provides a portable and repeatable runtime environment. The combined approach improves collaboration, deployment reliability .

